

ExcExtract

A 2G / 3G / 4G cell-naming automation tool — runs entirely in the browser

A web app built with [React](#) and [Vite](#) that runs entirely in the browser with no backend server. Its job: take a single Excel file containing network cell-site data, and generate a new Excel file from it with three worksheets — **2G**, **3G** and **4G** — automatically generating the cell names for every sector/carrier according to fixed naming rules.

1 — Overall project structure

App.jsx — Entry point: wires state to the UI, holds no logic of its own



useExcelGenerator.js — Hook that manages state and orchestrates the generation flow



excelUtils.js — Core logic: reading the file, listing its worksheets, validation, transformation, building the output

The UI itself is made of just two components:

Component	Role
UploadPanel.jsx	Picking the file, showing its name and size, and the generate / reset buttons
DownloadPanel.jsx	Showing the processing state and the resulting file with its download link

2 — Every file and function explained

App.jsx

The UI entry point. It contains no logic of its own — it simply takes the state and functions from `useExcelGenerator` and passes them down to the `UploadPanel` and `DownloadPanel` components.

useExcelGenerator.js — state and orchestration

Function	What it does
<code>useExcelGenerator()</code>	Holds the state: the selected file, its worksheet list and the chosen sheet, the generated files, the error message, and whether processing is currently running
<code>selectFile(file)</code>	Runs when a file is chosen — reads the workbook straight away, fills the worksheet picker, and preselects the first sheet that has all the required columns
<code>clearDownloads()</code>	Revokes the old file links (blob URLs) and then empties the list, so they don't pile up in browser memory between runs
<code>resetAll()</code>	Clears the selected file, the error message and the generated files — wired to the "Reset" button
<code>generateFiles(file)</code>	The main operation triggered by the "Generate workbook" button — broken down in the steps below

The steps of `generateFiles`, in order:

1	Checks that a file is selected and that it is an <code>xlsx</code> , <code>xls</code> or <code>csv</code> file
2	Clears previous results and errors, and enters the "processing" state
3	Waits a short moment (900 ms) purely to smooth out the user experience
4	Calls <code>createWorkbookWithNetworkTabs(file)</code> to perform the actual conversion
5	Puts the resulting file into the downloads list, or shows the error message if one occurred

UploadPanel.jsx — the upload interface

The `handleFileChange` function reads the file from the file input, passes it upward, then clears the input's value so the change event still fires if the user picks the same file again. The panel displays the file name and size, the worksheet picker, the error message if there is one, and the generate and reset buttons.

The worksheet picker lists every sheet in the uploaded workbook with its row count, and marks any sheet that cannot be converted with the number of columns it is missing. This matters because real workbooks often open on a summary or index sheet rather than the site data — the app preselects the first sheet that has all the required columns, and the user can switch to another one.

DownloadPanel.jsx — the result interface

A display-only component: it shows "processing", or an empty message at the start, or the list of generated files with a direct download link for each one.

3 — excelUtils.js — the heart of the project (conversion logic)

Core constants

Constant	What it holds
<code>NETWORK_FILE_NAMES</code>	The worksheet names to create: 2G, 3G, 4G
<code>TWO_G_SUFFIXES</code>	The 2G cell suffixes: 1, 2, 3, 5, 6, 7 — the number 4 is deliberately skipped
<code>NETWORK_SITE_NAME_REPLACEMENT</code>	The letter that replaces "GUL" in the site name: 2G becomes G, 3G becomes W, and 4G becomes L
<code>OUTPUT_COLUMNS</code>	The fixed column order in every generated worksheet: Cell Name, Region, Site Name, Site Id, Longitude, Latitude, Azimuth, M.Tilt, TRX Number, Tower Type, Tower Height
<code>FOUR_G_SUFFIXES_BASE</code>	The base 4G cell suffixes: A1, A2, A3, B1, B2, B3, C1, C2, C3
<code>FOUR_G_SUFFIXES_EXTENDED</code>	The same list, plus B4, B5, B6 when there is a 30 MHz carrier
<code>FOUR_G_SUFFIXES_L1800 / _L2100</code>	The suffix groups tied to a band — A1/A2/A3 belong to L1800, C1/C2/C3 to L2100 — dropped when that band is not deployed on the site

Handling column-name variations

Function	What it does
<code>normalizeHeaderName</code>	Lowercases the text and strips symbols, so different header spellings match (such as "Site Code" and "SiteCode")
<code>resolveColumnName</code>	Looks up whichever accepted alias actually exists in the uploaded file
<code>buildColumnMap</code>	Builds a map linking each logical field (Region, Site Code, Site Name...) to the real column name in the file
<code>getMissingColumns</code>	Determines which required columns are absent, so a clear error message can be shown to the user

Row transformation

Function	What it does
<code>extractSiteIdFromSiteName</code>	Extracts the last 4-digit number in the site name to use as the Site Id
<code>buildCellBaseName</code>	Replaces "GUL" in the site name with the letter matching the network — e.g. BNL-Site-GUL-3373 becomes BNL-Site-L-3373 for 4G
<code>hasThirtyMhzL800</code>	Checks whether the L800 BW column contains "30 MHz", to decide whether to use the extended 4G suffixes
<code>isBandMissing</code>	Treats a bandwidth cell as "band not deployed" when it is blank or contains just "-"
<code>getFourGSuffixes</code>	Picks the final 4G suffix list for one site: starts from the base or extended set, then drops A1/A2/A3 if L1800 BW is missing and C1/C2/C3 if L2100 BW is missing
<code>splitSectorValues</code>	Splits a per-sector cell such as 115/185/255 (or 115,185,255) into its three values; returns nothing if it is not a full set of three
<code>parseConfigGroups</code>	Parses a config cell into groups of per-sector values, accepting S5/5/5 0/0/0, 6/6/6 0/0/0, S555/000, S222 and S2/2/2 alike
<code>getTwoGCells</code> / <code>getThreeGCells</code> / <code>getFourGCells</code>	Build the cell list for one site on one network — the suffix, its sector, and its TRX Number — after applying that network's config and band rules
<code>getSectorValue</code>	Picks the one sector value matching a cell's position in its suffix list, cycling back to the first sector every three cells
<code>createOutputRow</code>	Builds one output row in the final layout defined by OUTPUT_COLUMNS, resolving Azimuth down to this cell's own sector
<code>getUniqueRowsByRegionAndSiteCode</code>	Removes duplicates from the input rows, keeping only the first row for each unique (Region + Site Code) combination

General helpers

Function	What it does
<code>getFileExtension</code> / <code>isExcelFile</code>	Check the uploaded file's extension and that it is a valid Excel file
<code>formatFileSize</code>	Converts the file size into a readable form (B, KB, MB, GB)
<code>cleanupDownloadUrls</code>	Releases blob URLs from browser memory

4 — The main operation: createWorkbookWithNetworkTabs

- 1 Reads the uploaded file and parses it using the `xlsx` (SheetJS) library
- 2 Takes **the worksheet the user picked** (falling back to the first one) and converts its data into rows
- 3 Verifies that all required columns are present — otherwise it throws an error listing the missing ones
- 4 Removes duplicate rows by (Region + Site Code)
- 5 For each network and each unique site, generates several rows with different cell names per the table below
- 6 Adds each network's rows as its own worksheet, and builds the final Excel file as a downloadable file

Cell generation rules per network:

Network	Cell count	Suffixes used
2G	up to 6 cells	-1 through -7 skipping -4, then trimmed by 2G Config
3G	up to 6 cells	-1 through -6, trimmed by 3G Config
4G – normal case	9 cells	A1, A2, A3, B1, B2, B3, C1, C2, C3
4G – with 30 MHz L800	12 cells	the list above + B4, B5, B6

The 4G list above is then trimmed per site, according to which bands that site actually has:

Condition on the source row	Effect on that site's 4G cells
L1800 BW is blank or "-"	A1, A2, A3 are dropped
L2100 BW is blank or "-"	C1, C2, C3 are dropped

So a site with neither L1800 nor L2100 produces only the B cells — 3 of them normally, or 6 when L800 BW is 30 MHz. 2G and 3G are unaffected by these rules.

2G and 3G cells come from the config columns

2G Config holds two TRX groups — one value per sector each, written as `S5/5/5 0/0/0`, `6/6/6 0/0/0` or the compact `S555/000`. The first group feeds cells -1, -2, -3 and the second feeds cells -5, -6, -7, and each cell's value becomes its TRX Number. A group of zeros means those cells do not exist, so the common `0/0/0` second group leaves only three 2G cells.

3G Config holds the carrier count per sector, written as `S222`, `S2/2/2` or `S111`. Cells -1, -2, -3 are the first carrier of each sector and -4, -5, -6 the second, so `2/2/2` produces all six cells while `1/1/1` produces only three. A sector with 0 carriers (as in `S101`) produces no cell of its own, and a config of - or a blank cell produces no rows for that network at all.

4G has no config column, so its rows leave TRX Number empty.

Azimuth, M.Tilt and TRX per sector

The source **Azimuth** and **M.Tilt** each hold all three sectors in one cell — for example **115/185/255**. Each generated cell takes only its own sector's value, following its position in the suffix list, and the sequence restarts at the first sector every three cells:

Cell	Azimuth written to the output row
...-1 / A1, B1, C1	115 (first sector)
...-2 / A2, B2, C2	185 (second sector)
...-3 / A3, B3, C3	255 (third sector)
...-5, ...-6, ...-7 / B4, B5, B6	115, 185, 255 again (the cycle restarts)

Values separated by commas (**325,60,150**) are handled the same way. If the cell does not hold exactly three values, it is copied through unchanged.

Every generated row otherwise keeps the same source data (region, coordinates, tower details) with a new cell name and a re-extracted Site Id. The final file is produced as: 2, 3, 4 G cell_naming.xlsx

5 — Quick workflow summary

1. Upload a single Excel file → **2.** Validate and match the columns → **3.** Remove duplicates by site → **4.** Expand every unique site into multiple rows with automatically generated cell names → **5.** Output one Excel file with three worksheets (2G, 3G, 4G), downloadable straight from the browser